

fdgl

rümeysa bilik

August 2026

Asymmetry Score

D_{asym} is the Randers-perturbed geodesic distance matrix (asymmetric: $D_{\text{asym}}[i, j] \neq D_{\text{asym}}[j, i]$ in general), and $\text{bln}[i, j]$ marks the real, directly constructed graph edges. The score is computed only over these direct neighbours, not all $n - 1$ pairs. For node i and each direct neighbour j with $\text{bln}[i, j] = \text{True}$:

$$\text{ratio}_{ij} = \frac{|D_{\text{asym}}[i, j] - D_{\text{asym}}[j, i]|}{D_{\text{asym}}[i, j] + D_{\text{asym}}[j, i]} \in [0, 1)$$

$\text{ratio}_{ij} = 0$ means the edge is perfectly symmetric ($D_{\text{asym}}[i, j] = D_{\text{asym}}[j, i]$); it approaches 1 only when one direction's distance collapses to 0 relative to the other.

$$\text{asym}_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \text{ratio}_{ij}, \quad \mathcal{N}(i) = \{j : \text{bln}[i, j] = \text{True}, j \neq i\}$$

$$\text{asym}_{\text{global}} = \frac{1}{n} \sum_{i=1}^n \text{asym}_i$$

asym_i is a *mean* over i 's direct neighbours, not a sum – this keeps the score in $[0, 1)$ and comparable across different values of k . Table 1 reports $\text{asym}_{\text{global}}$ for the swiss-roll dataset across both adjacency rules and several values of k .

	k=10	k=20	k=30	k=40	k=50
ε threshold	0.3204	0.3206	0.3209	0.2985	0.2942
k-nearest-neighbours	0.3212	0.3211	0.3211	0.3203	0.3210

Table 1: asymmetry scores of swiss roll for different numbers of neighbourhood

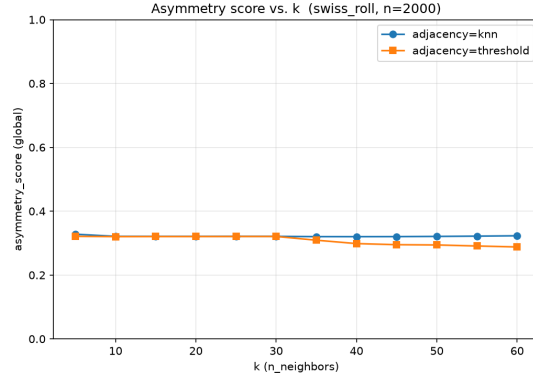


Figure 1: Graph of Asymmetry Score vs Number of Neighbours for Swiss Roll

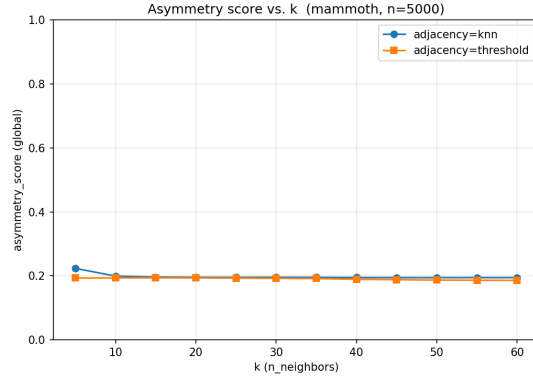


Figure 2: Graph of Asymmetry Score vs Number of Neighbours for Mammoth

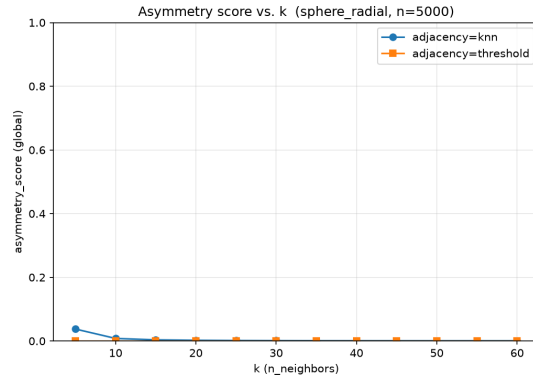


Figure 3: Graph of Asymmetry Score vs Number of Neighbours for Sphere with Radial Drifts

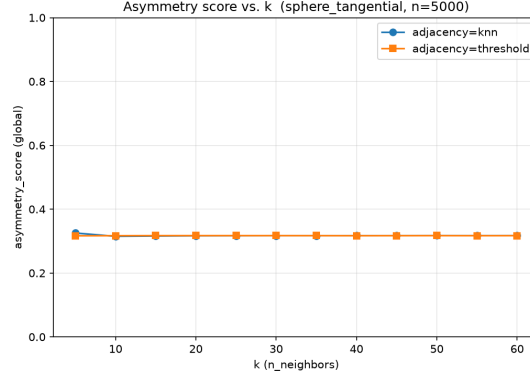


Figure 4: Graph of Asymmetry Score vs Number of Neighbours for Sphere with Tangential Drifts

Randers vs. Euclidean Force Toggles

In `randers_umap_fit`, the attraction, repulsion, and gravity terms can each independently use either the Randers-perturbed metric or the plain Euclidean one, via three booleans: `randers_attractive`, `randers_repulsive`, `randers_gravity`. When a term's flag is `True`, it uses $\rho_r = d + \langle B, y_j - y_i \rangle$ and $g_r = e + B$ (the drift B perturbs both the effective distance and direction); when `False`, it falls back to the plain Euclidean $\rho_e = d$, $g_e = e$, ignoring B entirely for that term. Gravity works the same way, but pulls node i toward its own virtual target $y_i + b_i$ rather than toward another node. The figures below sweep gravity strength under different combinations of these three flags.

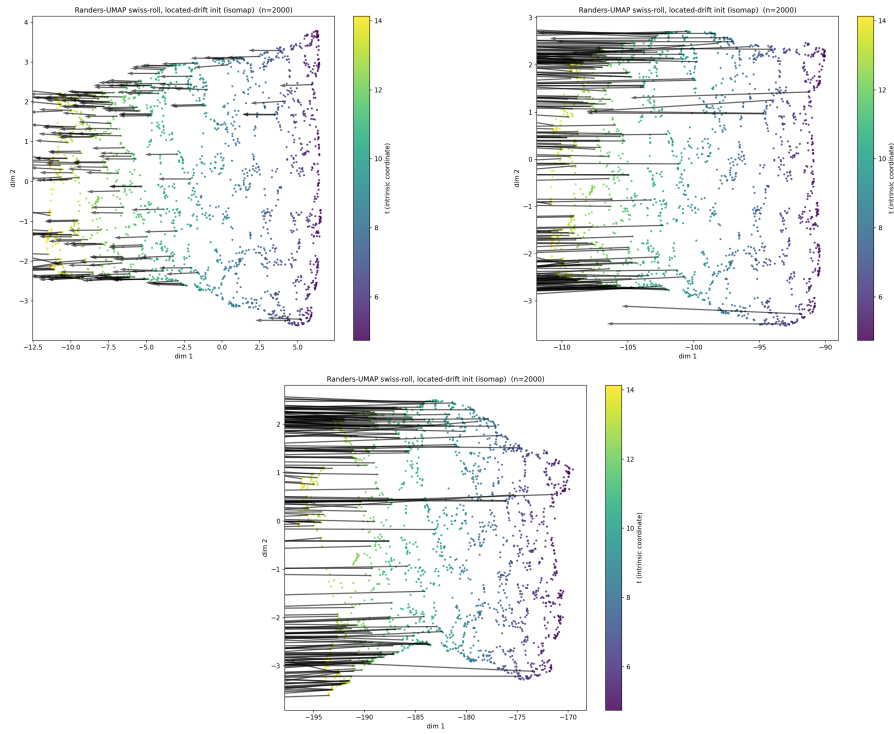


Figure 5: swiss roll embedding with gravity strength 0, 5, 10 with `grav_randers=True`, `attr_randers=False`, `rep_randers=False`

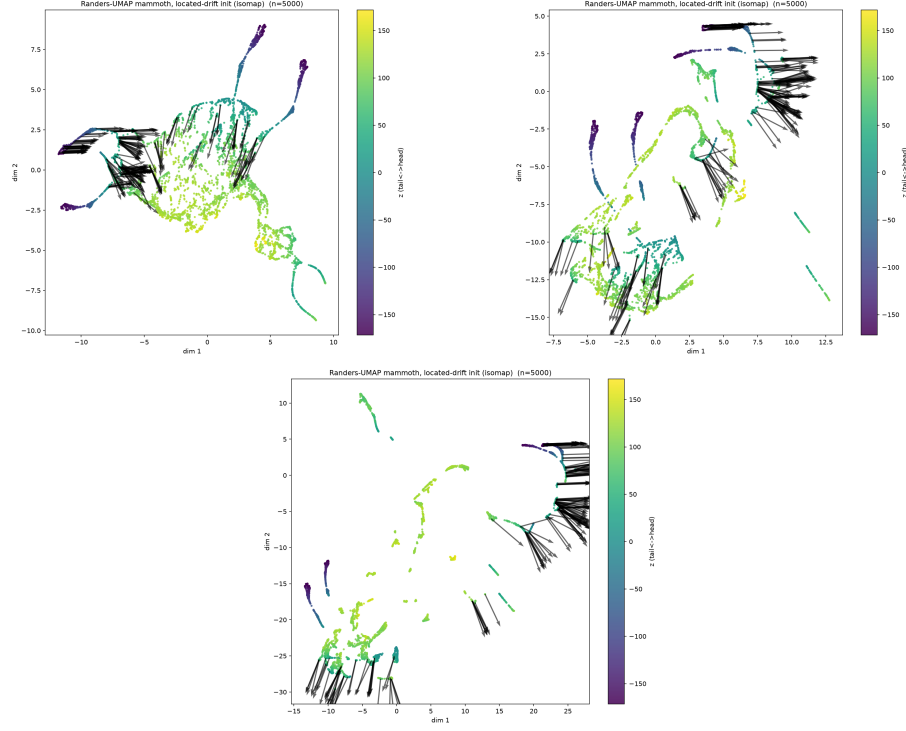


Figure 6: mammoth embedding with gravity strength 0, 5, 10 with `randers_gravity=True`, `randers_attractive=False`, `randers_repulsive=False`

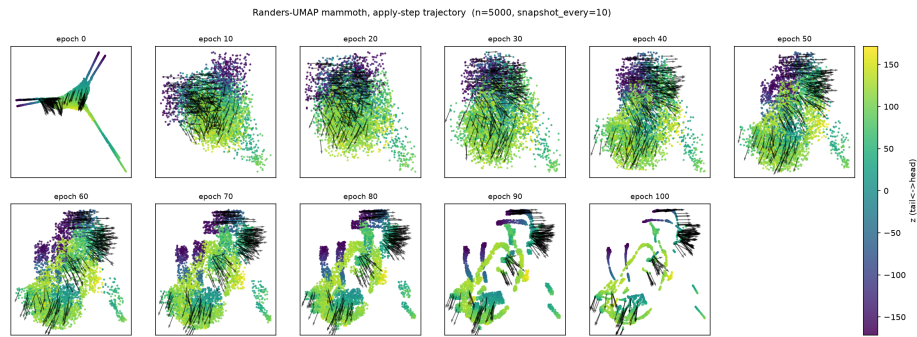


Figure 7: Snapshots of mammoth embedding for strength 5 gravity

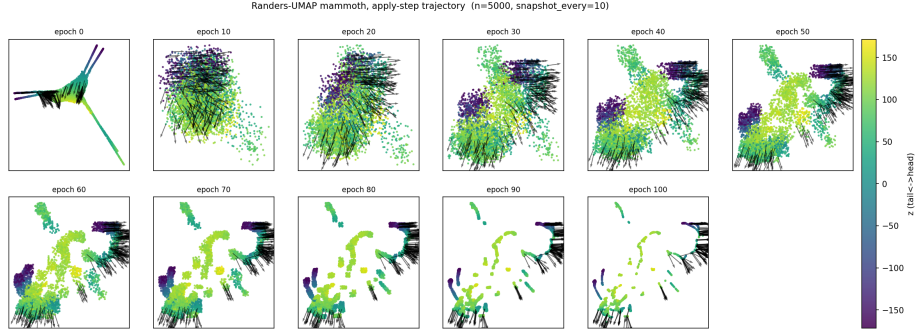


Figure 8: Snapshots of mammoth embedding for strength 10 gravity

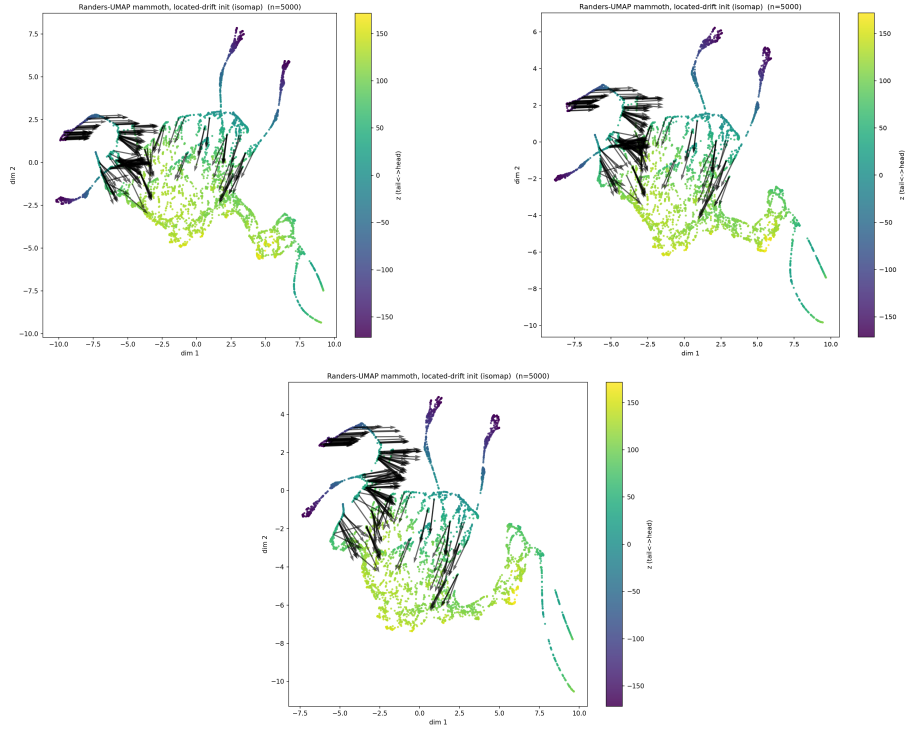


Figure 9: mammoth embedding with gravity strength 0, 5, 10 with `randers_gravity=False`, `randers_attractive=True`, `randers_repulsive=False`

Drift Magnitude Normalization

The drift vector B_i that feeds the Randers terms above is frozen once, at the "locate" step, before training. This section compares how its magnitude is treated afterwards: a fixed clipped constant (before) versus a live, embedding-relative rescaling (now, via `--normalize`). **Before: clip_delta-based, frozen for the whole run**

$$B_i^{(0)} = \hat{Y}_i^{\text{virt}} - \hat{Y}_i^{\text{real}}$$

Locate step: a single deterministic placement, computed once, before any training.

$$B_i = B_i^{(0)} \cdot \min\left(1, \frac{1 - \delta}{\|B_i^{(0)}\|}\right), \quad \delta = \text{clip_delta (default 0.01)}$$

$$\implies \|B_i\| \leq 1 - \delta \approx 1$$

A fixed, embedding-scale-independent constant.

$$B_i(t) = B_i \quad \forall t = 1, \dots, T$$

Both direction *and* magnitude are constant for the whole training run. **Now:**

`scale_B_fixed_by_knn_distance (--normalize)`

$$\hat{b}_i = \frac{B_i^{(0)}}{\|B_i^{(0)}\|}$$

Direction is still frozen, exactly as before – unaffected by everything below.

$$d_i^{(k)}(t) = \max_{j \in \text{kNN}_k(i, Y(t))} \|y_i(t) - y_j(t)\|$$

Distance to node i 's k -th nearest neighbour in the *live* embedding $Y(t)$.

$$B_i(t) = \frac{\|B_i^{(0)}\|}{d_i^{(k)}(t)} \cdot \hat{b}_i$$

Recomputed every epoch t . The magnitude is the *frozen located magnitude* divided by the live k -th-nearest-neighbour distance.

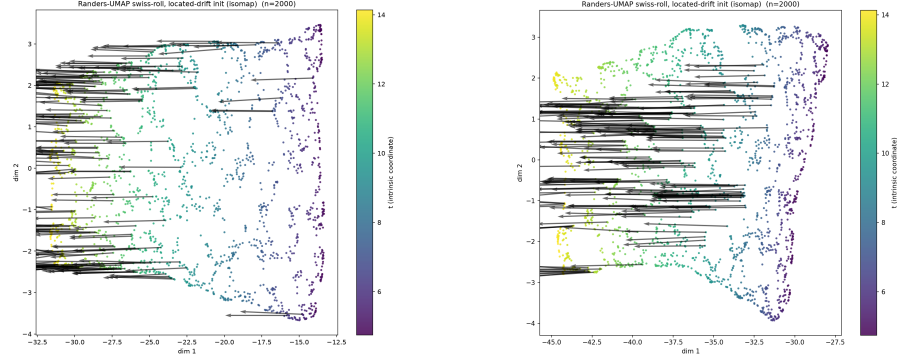


Figure 10: results of swiss roll embedding without normalization and with normalization (epochs 100)

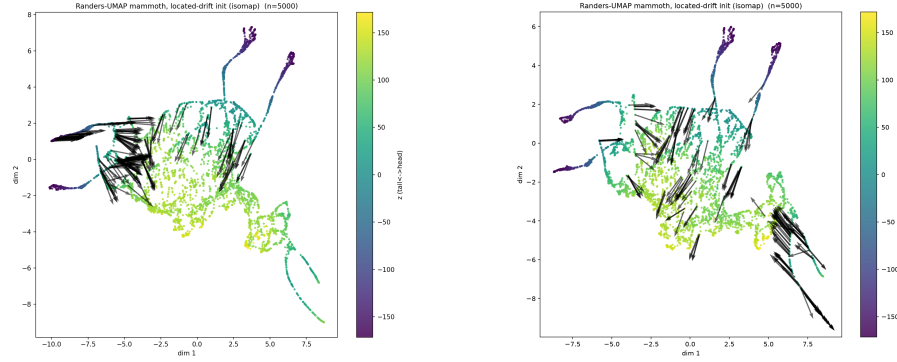


Figure 11: results of mammoth embedding without normalization and with normalization (epochs 50)

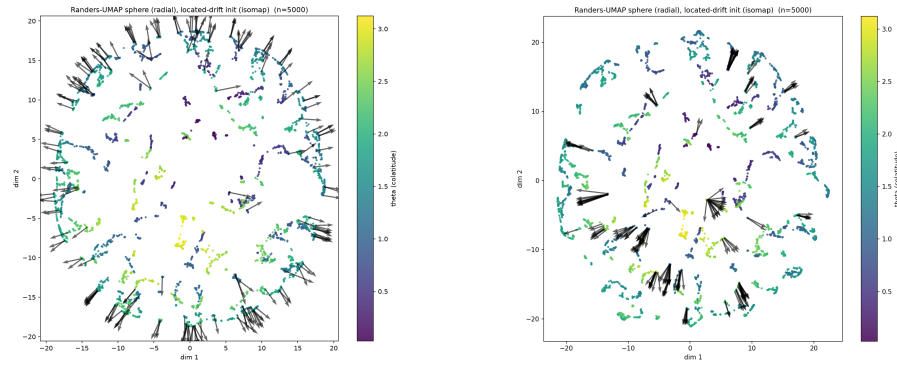


Figure 12: results of sphere with radial drifts embedding without normalization and with normalization (epochs 50)

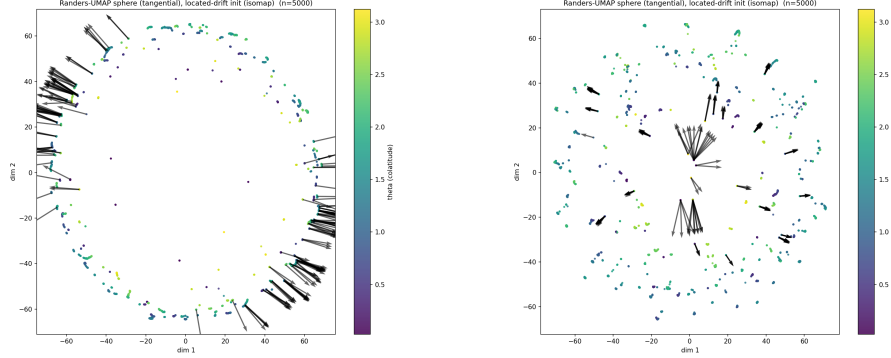


Figure 13: results of sphere with tangential drifts embedding without normalization and with normalization (epochs 50)

0.1 MNIST: Raw vs. PCA-First Pipelines

Motivation. MNIST offers a natural annotated ($n = 70,000$, 784-dimensional, 10-class) test bed beyond the synthetic manifolds used elsewhere in this report. Two ways of applying our method were suggested: (1) run the pipeline directly on the raw 784-dimensional pixel space, or (2) linearly project to a lower dimension first (van der Maaten’s classic t -SNE-preprocessing recommendation, ~ 50 dimensions) and run the pipeline on the projection. Both were implemented as a controlled pair, `embed_MNIST_raw.py` and `embed_MNIST_pca.py`, sharing every stage of the pipeline except the presence of the PCA step, so that any difference in the resulting embeddings can be attributed to the ambient dimensionality alone.

Shared pipeline. Let $X \in \mathbb{R}^{n \times 784}$ (raw) or $X_{\text{pca}} \in \mathbb{R}^{n \times d}$, $d = 50$ (PCA-reduced), denote the input to the following identical stages.

1. **Local asymmetric neighbourhood distance.** IsUMap’s own `distance_graph_generation` routine (unmodified, vendored library code) is run with $k = 30$ neighbours and per-point normalisation. It returns a dictionary R keyed by triples (i, j, k) , meaning “distance from j to k as measured within i ’s neighbourhood”; the genuine, directly measured distance from a point i to one of its own neighbours is stored exactly when $j = i$, with the third index giving the true target.
2. **Matrix construction.** A sparse asymmetric matrix is built by keeping only the $i = j$ entries of R and writing them as $D_{\text{sparse}}[i, k] = R[(i, i, k)]$, with every unmeasured pair left as $+\infty$. (An earlier version of this step wrote every (i, j, k) triple to position $[i, j]$ regardless of the $i = j$ condition; this landed real distances on the diagonal and unrelated triangulated values at the wrong indices, leaving $\sim 95\%$ of the matrix at a spurious literal zero. Since nearest-neighbour search always prefers the smallest value, the

resulting neighbour graph was pure noise — verified to have 0% overlap with the true nearest neighbours on a controlled synthetic test, and to be the direct cause of an embedding with no visible digit clusters regardless of epoch count. The fix above restores 100% overlap on the same test.)

3. **Directed completion.** D_{sparse} , still sparse ($\sim k$ finite entries per row), is completed into a dense, connected, still-asymmetric matrix $D_{\text{asym}} \in \mathbb{R}^{n \times n}$ via directed Dijkstra (`scipy.sparse.csgraph.shortest_path, directed=True`) — the same completion strategy used by `randers_bridge.compute_dist_matrix` elsewhere in this project.
4. **Embedding.** D_{asym} is passed directly to `randers.umap_fit` ($n_{\text{neighbors}} = 20$, 10 negative samples, 500 epochs, `use_drift=True`, $B_{\text{fixed}} = \text{None}$). The drift field B is therefore not fixed in advance but recovered live every epoch from D_{asym} 's own asymmetry via `compute_drift`, with `norm_mode="relative"` guaranteeing $\|b_i\| < 1$ by construction — no virtual points and no class-centroid targets are involved.

Where the two variants differ. `embed_MNIST_pca.py` inserts a single step before stage 1: $X \mapsto X_{\text{pca}} = \text{PCA}_d(X)$, $d = 50$ by default, and stage 1 is then run on X_{pca} instead of X . `embed_MNIST_raw.py` omits this step entirely and runs stage 1 directly on the full 784-dimensional X . No other stage, hyperparameter, or force toggle differs between the two scripts.

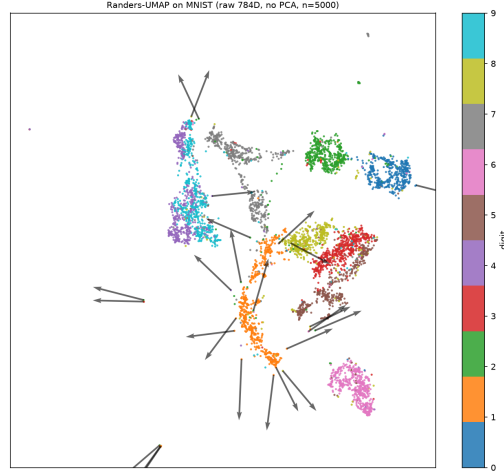


Figure 14: MNIST Embedding on 28x28-dim with 300 Epochs

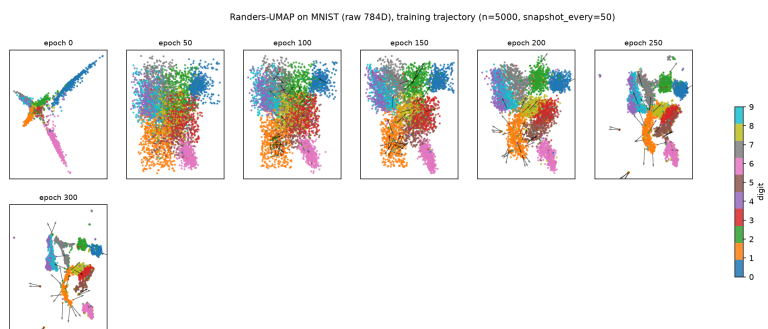


Figure 15: Snapshots of MNIST Embedding on 28x28-dim with 300 Epochs

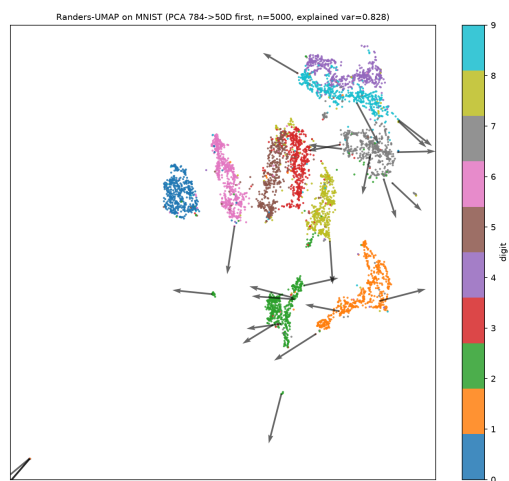


Figure 16: MNIST Embedding on 50-dim with 300 Epochs

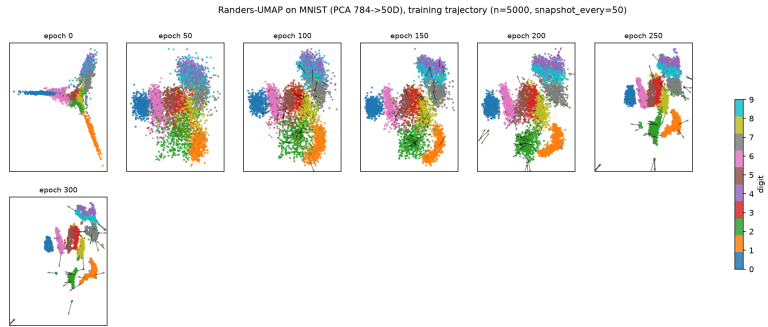


Figure 17: Snapshots of MNIST Embedding on 50-dim with 300 Epochs

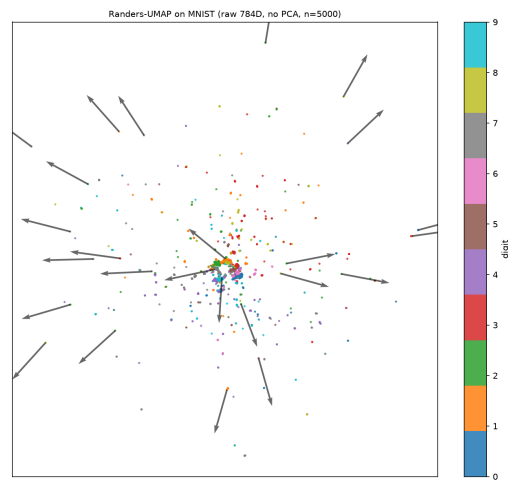


Figure 18: MNIST Embedding on 28x28-dim with 300 Epochs with gravity of strength 5

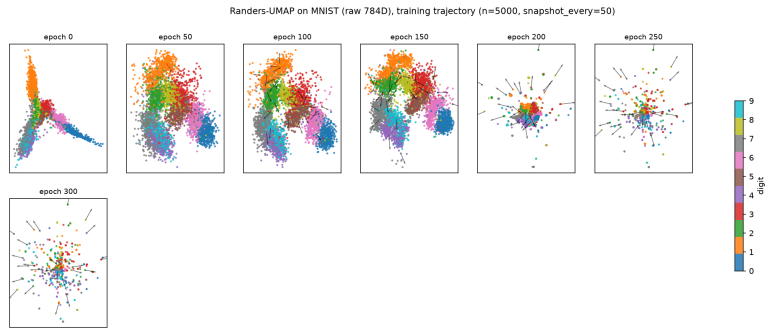


Figure 19: Snapshots of MNIST Embedding on 28x28-dim with 300 Epochs with gravity of strength 5

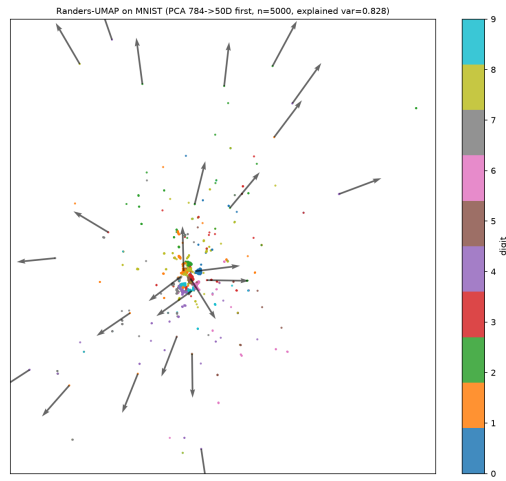


Figure 20: MNIST Embedding on 50-dim with 300 Epochs with gravity of strength 5

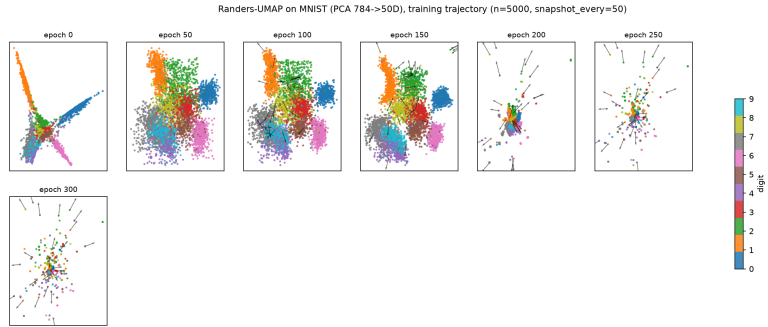


Figure 21: Snapshots of MNIST Embedding on 50-dim with 300 Epochs with gravity of strength 5

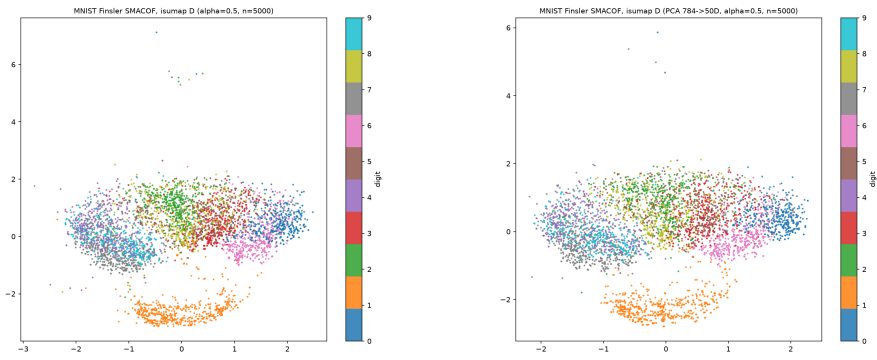


Figure 22: FinslerMDS results for MNIST Embedding without and with PCA

0.2 Wisconsin Breast Cancer Dataset

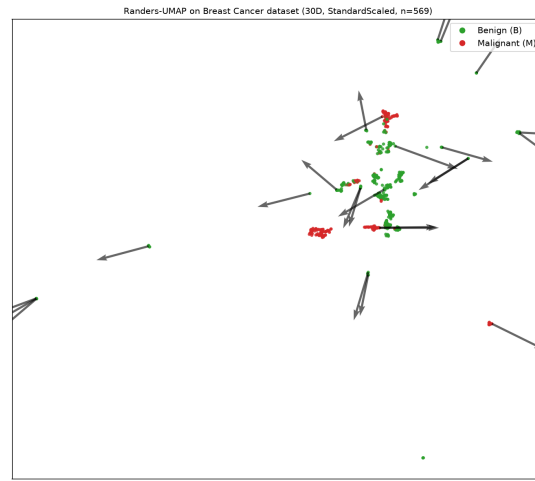


Figure 23: Breast Cancer Dataset Embedding with 100 Epochs

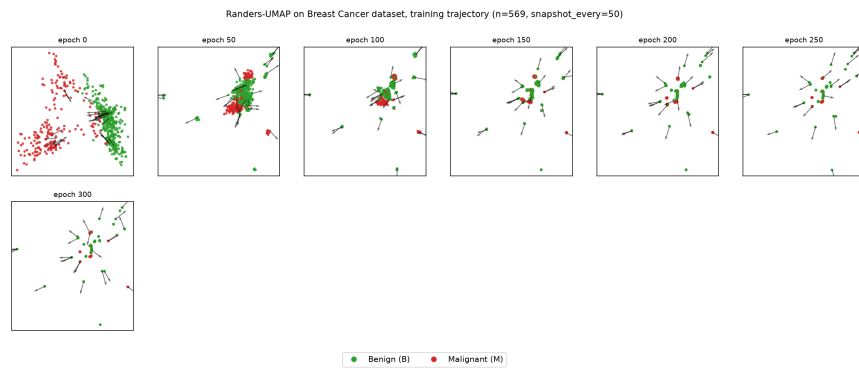


Figure 24: Breast Cancer Dataset Embedding Snapshots with 300 Epochs